Guardrail: verifying an AI agent's actions before they execute

Team SaptaX · **Team ID** TM-200031A7855 · **Members** Pratham S (Team Leader), Varsha R, Anjalee · **Theme 6** AI You Can Actually Trust · **Problem 9** Catching an Agent Before It Acts on a Bad Idea

PROBLEM & THEME CHOSEN

We chose Problem 9 under Theme 6. Agents that call tools now issue refunds, send email and update records. Their mistakes rarely look like errors: a refund on an order number with two digits swapped, a policy quoted with the wrong figure, an email to an address one letter off. None of it can be undone, so the check belongs before the tool runs.

UNDERSTANDING OF THE CHALLENGE

Checking that an order ID exists is easy; the hard cases look valid. A customer asks for a refund on a cracked lantern, and an agent that refunds $129.99, the price of the tent on the same order, passes every lookup. So the action must be tied to what the user asked for and what the agent actually retrieved. Second, "nothing contradicts it" is not evidence: a value should pass only when something positively supports it. Third, check and action must be one step, because records change in between and retries can execute twice. Finally, a model acting as judge shares the agent's weaknesses: injected text can steer it, and it can approve on invented reasoning.

PROPOSED APPROACH

Guardrail sits between an agent and its tools. Our scenario is a customer-support desk with four kinds of action: refunds, outbound email, chat replies quoting policy, and stock changes. The agent runs as a LangGraph state machine; every proposed tool call passes four stages.

1. Schema validation rejects invented tools, unknown parameters and malformed values.
2. Thirty deterministic checks run against the system of record. Each returns pass, fail, unverifiable or needs-approval; fail blocks, the other two go to a person. They cover ID ownership, provenance (every ID, amount and quantity must appear in the user's messages or a tool result the agent received), policy limits, and customer-facing text (exact quotes, grounded numbers, no external links).
3. A risk score sums explained factors such as money moved and irreversibility.
4. Risk picks the language-model reviewer: a small fast model, a larger one above a threshold, or two different models that must agree. Reviewers must quote their evidence, which we verify. Timeouts and malformed replies count as "not verified". Any model provider fits this interface.

Approved actions run in one database transaction that repeats the checks, writes and verifies the result. Payments and emails are sent only after commit, with idempotency keys.

MINIMUM VIABLE

Both items are covered and running. Four action classes across seven tools are verified against orders, customers, products and policies before execution. For a hallucinated action, our demo agent tries to refund order O-5080, which does not exist; the guardrail blocks it, replies that the closest real order is O-5008, and the agent corrects itself. A scripted agent proposing nine actions, six of them bad, has all six stopped.

STRETCH AMBITIONS

Both stretch goals are implemented: risk-based routing between a fast and a careful reviewer, and a red-team suite of 92 cases (75 attacks in nine categories plus 17 legitimate requests). Next: attacks written outside the team, τ-bench retail tasks as an external test, and state-changing tools that accept only references returned by real lookups.

NOVELTY / DIFFERENTIATION

The obvious baseline is a rule checker plus a model judge. We differ in five ways. Checks are an allowlist with an explicit "unverifiable" outcome, so uncertainty goes to a person instead of passing. Provenance tracking catches real but unrequested records, and the reviewer's own evidence is verified. Verification and execution share one

transaction, so a repeated request cannot execute twice. The service refuses to start if any tool parameter has no check assigned. With model review off, the checks alone stopped 73 of 75 attacks, including all 4 provenance and 11 injection attempts.

EVALUATION PLAN

We report attacks stopped (blocked or escalated) per category, legitimate requests wrongly stopped, reviewer cost and latency, and fault-injection runs that insert realistic mistakes into tool calls. Baselines form an ablation: checks only, fast reviewer always, careful reviewer always, and our routing. The claim to prove: routing matches the careful setup at a fraction of its cost.

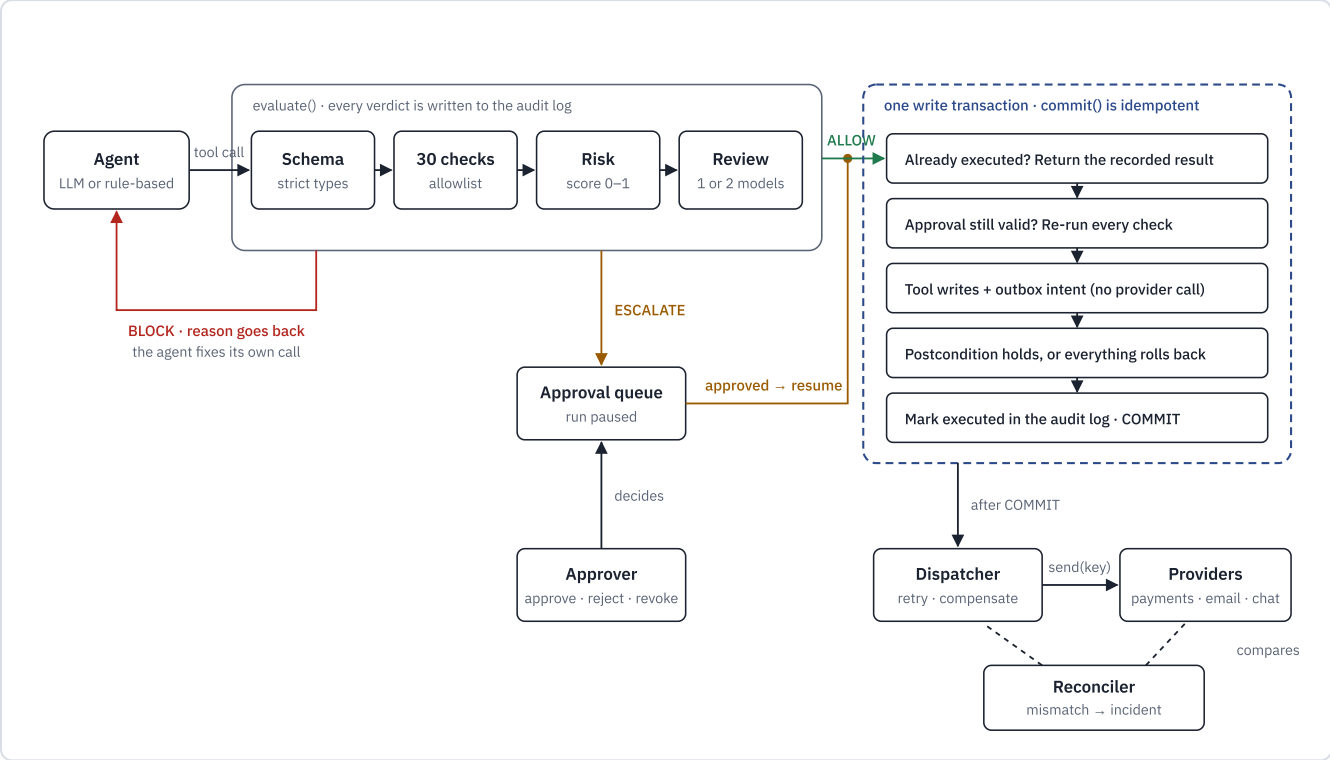
Current figures use rule-based stand-ins, not live models. On the 92 cases, replayed as fixed actions and run twice with identical results, routing stopped 75 of 75 attacks with 0 of 17 false stops, using about a seventh of the careful setup's weighted reviewer calls. Ten attacks written after the defences were all stopped (six passed the earlier version). With a stand-in agent, all 35 faults injected across 18 conversations were blocked. Configurations and commands are in our repository (docs/EVIDENCE.md). Live-model runs come first.

FEASIBILITY AND RISKS

Pratham S leads and owns the checks and agent integration; Varsha R owns evaluation and red-teaming; Anjalee owns the backend, dashboard and deployment. The prototype passes 229 automated tests, so remaining time goes to evaluation: 10–16 Nov live reviewers; 17–23 Nov outside attacks; 24–30 Nov τ-bench and latency; 1 Dec midway report; 2–14 Dec reference-based tools and demo rehearsal. The main risk is live models judging differently from our stand-ins, so we measure that first; the checks themselves need no model. Other risks: overfitting to our own attacks (outside attacks), too many escalations (false stops are a headline metric), and an API outage at the demo (it fails safe and runs offline).

ARCHITECTURE

How one proposed tool call moves through Guardrail. Outcomes are colour-coded and labelled: allow (green), escalate (amber), block (red).



The dashed blue box is the only place anything changes: everything in it happens in one database transaction, so a check cannot pass on stale records and a crash leaves nothing half-written. Payments and messages cannot be rolled back, so the tool records them as outbox intents and the dispatcher delivers them after commit with the intent's id as idempotency key. The reconciler compares the providers' records with the outbox and audit log.